

Missing data

KING'S
College
LONDON

Social and Cultural Analytics (7AAVBCS5)

Dr Mark J. Hill · mark.j.hill@kcl.ac.uk

2026-10-19



Lecture Plan

- **Part I: Missing data**
 - What counts as missing
 - `NaN`, `None`, and `.isna()`
 - Why pandas can be quiet about gaps
- **Part II: Missingness in London cultural data**
 - Counts and percentages
 - Website, postcode, and address fields
 - Missingness as evidence
- **Part III: Strategies**
 - Delete, fill, or keep
 - Missingness indicators
 - Preserving the original data
- **Part IV: Joining context**
 - How borough context gets attached
 - Checking whether missingness is patterned
- **Part V: Live coding**

Key Learning Outcomes

By the end of this week you will be able to:

- identify missing values in pandas
- quantify missingness with counts and percentages
- explain why missingness may be meaningful
- choose a defensible missing-data strategy
- create missingness indicators
- join venue records to borough context
- interpret missingness with denominators and caution

Review

What we learned last week

- Data frames store rows and columns
- One row should describe one thing
- `pd.read_csv()` loads CSV files
- `.info()`, `.head()`, `.shape`, `.columns`, and `.describe()` inspect structure
- Square brackets select columns
- `.loc[]` selects by label
- `.iloc[]` selects by position
- Masks filter rows
- `to_csv(index=False)` saves data without writing row labels

This week: what happens when a table has gaps

Part I: Missing Data

Missing data

What is missing data?

- a value that should logically exist, but does not
- a blank in a public record
- a field skipped by a survey respondent
- a place that cannot be joined to another geography because a postcode is absent
- an unknown value created by earlier data collection choices

Why it matters

- summaries may use fewer rows than you think
- comparisons may exclude particular places or groups
- missingness can reveal how the dataset was made

Why values go missing

Cause	Example	Question to ask
Recording error	a field was left blank during data entry	Is the error scattered or concentrated?
Non-response	someone does not answer an income question	Who is more likely not to answer?
Impossible to record	a historical source no longer exists	What historical process produced the gap?
By design	a field does not apply to some rows	Is this really missing, or not applicable?

Missing values in pandas

```
1 import numpy as np
2 import pandas as pd
3
4 example = pd.Series(["website", None, "website", np.nan])
5 example
```

```
0    website
1         NaN
2    website
3         NaN
dtype: str
```

```
1 example.isna()
```

```
0    False
1     True
2    False
3     True
dtype: bool
```

In pandas, missing values often appear as `NaN` or `None`. Use `.isna()` to detect both.

NaN, None, and empty strings

Value	Plain meaning	How to detect it
NaN	missing or undefined numeric value	<code>.isna()</code>
None	no Python value supplied	<code>.isna()</code>
<code>""</code>	an empty string, not automatically missing	compare to <code>""</code> or clean first
<code>0</code>	a real value, unless the codebook says otherwise	do not treat as missing by default

Missing values are not the same as zeros or empty text. Check the codebook and the data source.

Do not compare to missing values

```
1 print(np.nan == np.nan)
2 print(example == np.nan)
```

```
False
0    False
1    False
2    False
3    False
dtype: bool
```

```
1 example.isna()
```

```
0    False
1     True
2    False
3     True
dtype: bool
```

Never test for missing values with `== np.nan`. Use `.isna()`.

pandas can be quiet about gaps

```
1 ages = pd.Series([23, 45, np.nan, 31])
2
3 print("mean:", ages.mean())
4 print("based on:", ages.notna().sum(), "of", len(ages), "values")
```

```
mean: 33.0
based on: 3 of 4 values
```

pandas summary methods usually skip missing values by default. That can be useful, but it also means nothing may warn you that a result uses fewer rows than the full dataset.

The practical issue

Missing data affects every later step:

- summary statistics may use only complete values
- maps may exclude records without usable place information
- joins may fail when the shared key is missing or inconsistent
- group comparisons may over-represent better documented groups
- deletion can remove the cases you most need to understand

Part II: Missingness in London Cultural Data

The venue records

```
1 venues = pd.read_csv("../data/london_cultural_venues_2023.csv")
2
3 venues[[
4     "name", "venue_type", "borough_name", "postcode",
5     "website", "latitude", "longitude"
6 ]].head()
```

	name	venue_type	borough_name	postcode	website	latitude	longitude
0	DAGENHAM RM10	Artists workspaces	Barking and Dagenham	NaN	https://www.arbeit.org.uk/dagenham	51.549708	0.156633
1	Ice House Court	Artists workspaces	Barking and Dagenham	NaN	www.bowarts.org www.artiststudiofinder.org ...	51.531488	0.075994
2	River Mills Studios (Bow Arts)	Artists workspaces	Barking and Dagenham	Barking	https://bowarts.org/studios/riverside-mills-st...	51.532525	0.075726
3	Vue Dagenham	Cinemas	Barking and Dagenham	RM9 6UQ	NaN	51.532095	0.135759
4	Bow Arts	Creative workspaces	Barking and Dagenham	Abbey Road	www.bowarts.org www.artiststudiofinder.org ...	51.531410	0.077946

First check: where are the gaps?

```
1 missing_summary = pd.DataFrame({
2     "missing": venues.isna().sum(),
3     "missing_%": (venues.isna().mean() * 100).round(1),
4 })
5
6 missing_summary[missing_summary["missing"] > 0].sort_values(
7     "missing", ascending=False
8 )
```

	missing	missing_%
postcode	3042	68.6
address_2	2217	50.0
website	760	17.1
address_1	440	9.9
address	355	8.0
uprn	331	7.5
easting	8	0.2
northing	8	0.2
ward_code	6	0.1
ward_name	6	0.1

Counts and percentages

Counts answer:

How many values are missing?

```
1 venues["website"].isna().sum()
```

```
np.int64(760)
```

A count without a denominator is hard to interpret. Report both.

Percentages answer:

How large is the problem?

```
1 venues["website"].isna().mean() * 100
```

```
np.float64(17.14027965719441)
```

What is missing in this dataset?

Field	What the gap affects
website	online visibility and discoverability
postcode	address reliability and joining to other geographies
address_1, address_2	how precisely the venue can be located or checked
uprn	linking to property-level records
ward_name, ward_code	ward-level comparisons

Missingness as a research question

For this file, missing data suggests questions such as:

- Which venue types are easiest to find online?
- Which boroughs have weaker website coverage in the dataset?
- Are address gaps concentrated in particular records?
- Does missingness reflect the venue, the data source, or the cleaning process?



Note

Missingness is sometimes the finding, not just an obstacle before the finding.

A missingness indicator

```
1 venues = venues.copy()
2 venues["has_website"] = venues["website"].notna()
3 venues["missing_postcode"] = venues["postcode"].isna()
4
5 venues[["name", "venue_type", "has_website", "missing_postcode"]].head()
```

	name	venue_type	has_website	missing_postcode
0	DAGENHAM RM10	Artists workspaces	True	True
1	Ice House Court	Artists workspaces	True	True
2	River Mills Studios (Bow Arts)	Artists workspaces	True	False
3	Vue Dagenham	Cinemas	False	False
4	Bow Arts	Creative workspaces	True	False

A missingness indicator turns a gap into a variable we can analyse.

Is website missingness patterned?

```
1 website_by_type = (  
2     venues.groupby("venue_type")  
3     .agg(  
4         venues=("venue_id", "size"),  
5         website_share=("has_website", "mean"),  
6     )  
7 )  
8  
9 website_by_type["website_share"] = (  
10     website_by_type["website_share"] * 100  
11 ).round(1)  
12  
13 website_by_type.sort_values("website_share").head(10)
```

	venues	website_share
venue_type		
Libraries	347	0.0
Cinemas	123	10.6
Jewellery design	117	47.9
LGBTQ venues	50	54.0
Music nightclubs	133	66.9
Music venues all	371	74.4
Arts centres	26	92.3
Theatre	266	92.9
Music venues grassroots	132	93.9
Fashion and design	80	95.0

Counts by venue type

```
1 venues["venue_type"].value_counts().head(10)
```

```
venue_type
Archives          544
Music venues all   371
Libraries         347
Commercial galleries 300
Artists workspaces 270
Theatre           266
Museums and public galleries 253
Dance rehearsal studios 252
Theatre rehearsal studios 217
Dance performance venues 196
Name: count, dtype: int64
```

Small groups can produce dramatic percentages. Always check the denominator before interpreting a percentage.

Borough differences

```
1 website_by_borough = (  
2     venues.groupby("borough_name")  
3     .agg(  
4         venues=("venue_id", "size"),  
5         website_share=("has_website", "mean"),  
6         postcode_missing=("missing_postcode", "mean"),  
7     )  
8 )  
9  
10 website_by_borough[["website_share", "postcode_missing"]] = (  
11     website_by_borough[["website_share", "postcode_missing"]] * 100  
12 ).round(1)  
13  
14 website_by_borough.sort_values("website_share").head(10)
```

	venues	website_share	postcode_missing
borough_name			
Havering	22	40.9	95.5
Redbridge	29	48.3	79.3
Bromley	47	57.4	70.2
Enfield	48	58.3	83.3
Hillingdon	49	61.2	69.4
Sutton	24	62.5	70.8
Kingston upon Thames	24	62.5	79.2
Barnet	55	63.6	67.3
Bexley	36	63.9	86.1

	venues			website_share	postcode_missing		
	borough_name						
	Croydon	58	67.2			77.6	

What to do when you find missing data

Ask three questions before fixing anything:

1. **How much is missing?** Count it, calculate percentages, and check which columns are affected.
2. **Why might it be missing?** Look for patterns by borough, venue type, source, or collection process.
3. **What is your goal?** Description, mapping, prediction, and interpretation may need different choices.



Note

Document the missingness and document what you did about it.

How much is missing?

```
1 missing_summary = (  
2     venues.isna()  
3     .sum()  
4     .to_frame("missing")  
5 )  
6 missing_summary["missing_%"] = (  
7     missing_summary["missing"] / len(venues) * 100  
8 ).round(1)  
9  
10 missing_summary.sort_values("missing", ascending=False).head(8)
```

	missing	missing_%
postcode	3042	68.6
address_2	2217	50.0
website	760	17.1
address_1	440	9.9
address	355	8.0
uprn	331	7.5
easting	8	0.2
northing	8	0.2

Counts show the scale. Percentages show the proportion. You usually need both.

Why might it be missing?

Pattern	Possible interpretation
scattered across rows	data entry or source integration problem
concentrated in one venue type	documentation differs by sector
concentrated in one borough	local data source or coverage issue
concentrated in one field	the field may not have been collected consistently

Part III: Strategies

Strategy 1: Deletion

```
1 print("rows before:", len(venues))  
2 print("rows after dropping any missing value:", len(venues.dropna()))
```

```
rows before: 4434  
rows after dropping any missing value: 774
```

Deletion is simple, but it can change the dataset in ways you did not intend.

Deletion: pros and cons

Useful when:

- only a few rows are affected
- the analysis genuinely requires a complete field
- the missingness seems unrelated to the question

Risky when:

- many rows disappear
- missingness is patterned by place or venue type
- the removed rows are culturally or analytically important

Deletion with a reason

```
1 with_websites = venues.dropna(subset=["website"])
2
3 print("rows before:", len(venues))
4 print("rows with website:", len(with_websites))
```

```
rows before: 4434
rows with website: 3674
```

Dropping rows can be defensible when the analysis specifically requires that field.

Say what you dropped, why you dropped it, and how many rows remain.

Strategy 2: Filling values

```
1 labelled = venues.copy()
2 labelled["website_status"] = labelled["website"].fillna("No website recorded")
3
4 labelled["website_status"].head()
```

```
0          https://www.arbeit.org.uk/dagenham
1  www.bowarts.org  www.artiststudiofinder.org ...
2  https://bowarts.org/studios/riverside-mills-st...
3                               No website recorded
4  www.bowarts.org  www.artiststudiofinder.org ...
Name: website_status, dtype: str
```

This is not the same as discovering the missing websites. It labels the gap.

Filling can make data look more complete than it really is. Use clear labels when you are recording unknownness rather than replacing a value.

Imputation means filling in values

Imputation replaces missing values with plausible substitutes.

Common examples:

- numeric mean or median
- most common category
- group average within a borough or venue type
- model-based estimate

Imputed values are estimates. They are not newly discovered facts.

Strategy 3: Keep and handle

```
1 summary = venues.groupby("venue_type").agg(  
2     venues=("venue_id", "size"),  
3     websites_recorded=("has_website", "sum"),  
4     website_share=("has_website", "mean"),  
5 )  
6 summary["website_share"] = (summary["website_share"] * 100).round(1)  
7 summary.sort_values("website_share").head()
```

	venues	websites_recorded	website_share
venue_type			
Libraries	347	0	0.0
Cinemas	123	13	10.6
Jewellery design	117	56	47.9
LGBTQ venues	50	27	54.0
Music nightclubs	133	89	66.9

Keep the original field, add indicators, and report what each number is based on.

Keep and handle: why it often fits cultural data

Keeping missing values allows you to:

- preserve the original record
- compare different handling choices
- analyse missingness as a pattern
- avoid inventing values for uncertain cultural fields
- report clearly how many observations support each claim

MCAR, MAR, MNAR

Type	Plain meaning	Example
MCAR	missing for reasons unrelated to the data	a random file transfer failure
MAR	missingness relates to observed fields	website missingness varies by venue type
MNAR	missingness relates to the missing value itself	venues without websites are less likely to have websites recorded

The labels are less important than the reasoning: what pattern could have produced the gap?

MCAR: missing completely at random

Plain English: the gap has no relationship to the values in the dataset.

Example:

- a file transfer error randomly deletes a small number of website entries
- every borough and venue type has roughly the same chance of being affected

Implication: deletion is less likely to bias comparisons, although it still reduces data.

MAR: missing at random

Plain English: missingness relates to something we can observe.

Example:

- website coverage differs by `venue_type`
- postcode missingness differs by `borough_name`
- one source file supplied fuller address fields than another

Implication: analyse the pattern before choosing a handling strategy.

MNAR: missing not at random

Plain English: missingness relates to the missing value itself or to something we cannot observe.

Examples:

- venues with limited online presence are less likely to have websites recorded
- groups with lower institutional visibility have thinner public records
- historical records are absent because someone never collected them

Implication: simple deletion or filling can create serious bias.

Missingness and ethics

When cultural data is missing, ask:

1. Whose venues, institutions, or practices become harder to find?
2. Does the gap come from the cultural world, the data source, or our cleaning choices?
3. What would deletion remove from the analysis?
4. What would filling the gap pretend to know?

Cultural significance of missing data

Missing data can reflect the history of collection and classification.

In cultural datasets, gaps may show:

- which organisations maintain public-facing records
- which places are easier to map, search, or count
- whose activity enters official infrastructure lists
- which categories fit the data model and which do not

Missingness as evidence

```
1 missing_by_type = (  
2     venues.assign(missing_website=venues["website"].isna())  
3     .groupby("venue_type")  
4     .agg(  
5         venues=("venue_id", "size"),  
6         missing_websites=("missing_website", "sum"),  
7         missing_website_pct=("missing_website", "mean"),  
8     )  
9 )  
10  
11 missing_by_type["missing_website_pct"] = (  
12     missing_by_type["missing_website_pct"] * 100  
13 ).round(1)  
14  
15 missing_by_type.sort_values("missing_website_pct", ascending=False).head(10)
```

	venues	missing_websites	missing_website_pct
venue_type			
Libraries	347	347	100.0
Cinemas	123	110	89.4
Jewellery design	117	61	52.1
LGBTQ venues	50	23	46.0
Music nightclubs	133	44	33.1
Music venues all	371	95	25.6
Arts centres	26	2	7.7
Theatre	266	19	7.1
Music venues grassroots	132	8	6.1

	venues			missing_websites	missing_website_pct	
	venue_type					
	Fashion and design	80	4	5.0		

Part IV: Joining Context

Why join tables?

The venue file has one row per venue.

The borough context file has one row per borough.

A join lets us attach borough-level context to each venue record:

- population
- income
- PTAL score
- borough-level venue counts
- borough website share

The shared key

```
1 venues[["name", "borough_name"]].head()
```

	name	borough_name
0	DAGENHAM RM10	Barking and Dagenham
1	Ice House Court	Barking and Dagenham
2	River Mills Studios (Bow Arts)	Barking and Dagenham
3	Vue Dagenham	Barking and Dagenham
4	Bow Arts	Barking and Dagenham

```
1 borough_context = pd.read_csv(  
2     "../..data/london_cultural_infrastructure_2023_borough_context.csv"  
3 )  
4  
5 borough_context[["borough_name", "population_2015", "ptal_score_2014"]].head()
```

	borough_name	population_2015	ptal_score_2014
0	Westminster	230550.0	6.5
1	Camden	230300.0	5.7
2	Hackney	262700.0	4.9
3	Southwark	308650.0	4.9
4	Tower Hamlets	284300.0	5.0

The shared column is `borough_name`. That is the key used to match rows.

A left join

```
1 venues_with_context = venues.merge(  
2     borough_context[[  
3         "borough_name",  
4         "population_2015",  
5         "median_household_income_2012_13",  
6         "ptal_score_2014",  
7         "website_share",  
8     ]],  
9     on="borough_name",  
10    how="left",  
11 )  
12  
13 venues.shape, venues_with_context.shape
```

```
((4434, 22), (4434, 26))
```

A left join keeps all rows from the left table, then adds matching columns from the right table.

Always check a join

```
1 print("rows before:", len(venues))  
2 print("rows after: ", len(venues_with_context))  
3 print("missing context:", venues_with_context["ptal_score_2014"].isna().sum())
```

```
rows before: 4434  
rows after:  4434  
missing context: 0
```

A join can silently multiply rows or leave unmatched rows. Check row counts and missing joined fields.

Context and missingness

```
1 venues_with_context["high_ptal"] = (  
2     venues_with_context["ptal_score_2014"] >= 5  
3 )  
4  
5 (pd.crosstab(  
6     venues_with_context["high_ptal"],  
7     venues_with_context["has_website"],  
8     normalize="index"  
9 ) * 100).round(1)
```

has_website	False	True
high_ptal		
False	20.4	79.6
True	13.7	86.3

What the join adds

Before the join, we could ask:

- Which venues have websites recorded?
- Which venue types have weaker website coverage?
- Which boroughs have more missing postcodes?

After the join, we can also ask:

- Does missingness vary with borough-level transport accessibility?
- Does missingness vary with borough-level income?
- Do borough summaries agree with venue-level records?

Part V: Live Coding

Exercise 1: quantify missingness

1. Load `london_cultural_venues_2023.csv`
2. Use `.info()` to inspect the table
3. Build a missingness summary with counts and percentages
4. Identify the three columns with the most missing values
5. Write one sentence about why each gap might matter

Exercise 2: missingness indicators

1. Create `has_website`
2. Create `missing_postcode`
3. Compare website coverage by `venue_type`
4. Compare postcode missingness by `borough_name`
5. Check denominators before interpreting percentages

Exercise 3: join borough context

1. Load `london_cultural_infrastructure_2023_borough_context.csv`
2. Join it to the venue data using `borough_name`
3. Check row counts before and after
4. Check whether any joined context is missing
5. Use the joined data to ask whether website coverage varies by PTAL

Before next week

1. Complete `w04_worksheet.ipynb` in the workshop
2. Keep the original data intact when making derived columns
3. In your notes, write one paragraph about missingness in your own dataset



Note

The quiz covering this week's material is in the Week 4 content folder.

